EXPERIMENT 4: INFORMATION RETRIEVAL USING TF-IDF AND COSINE SIMILARITY

AIM:

To develop an information retrieval system using the 20 Newsgroups dataset by preprocessing text, removing English stop words, converting documents into TF-IDF vectors, and retrieving the most relevant documents for the query “space shuttle launch” using cosine similarity.

ALGORITHM:

1. Import the required libraries, including NLTK, Pandas, scikit-learn, and cosine similarity utilities.

2. Download the required NLTK resources and load the 20 Newsgroups training dataset, excluding headers, footers, and quoted text.

3. Store the dataset text in a Pandas DataFrame and inspect the sample records.

4. Obtain the English stop-word list and extract stop words from the document text.

5. Define a preprocessing function to convert text to lowercase, remove punctuation and non-alphanumeric characters as required, tokenize the text, and remove stop words.

6. Apply the preprocessing function to create a cleaned-text column.

7. Use TfidfVectorizer to convert the cleaned documents into TF-IDF feature vectors.

8. Enter the query “space shuttle launch” and preprocess it using the same function.

9. Transform the query into a TF-IDF vector and calculate cosine similarity with the document vectors.

10. Sort the similarity scores and display the top five most relevant documents.

11. Tokenize sample documents and display their part-of-speech tags.

```python
# EXPERIMENT-3B Muthuraj(241801176)

# Step 1
import nltk
import pandas as pd
import string
import re

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')

newsgroups_train = fetch_20newsgroups(subset='train')

df = pd.DataFrame({'text': newsgroups_train.data})

print(df.head(2).to_markdown())

newsgroups_data = fetch_20newsgroups(
    subset='train',
        remove=('headers', 'footers', 'quotes')
        )
```

...

	text
0	From: lerxst@wam.umd.edu (where's my thing) Subject: WHAT car is this!? Nntp-Posting-Host: rac3.wam.umd.edu Organization: University of Maryland, College Park Lines: 15 I was wondering if anyone out there could enlighten me on this car I saw the other day. It was a 2-door sports car, looked to be from the late 60s/ early 70s. It was called a Bricklin. The doors were really small. In addition, the front bumper was separate from the rest of the body. This is all I know. If anyone can tellme a model name, engine specs, years of production, where this car is made, history, or whatever info you have on this funky looking car, please e-mail. Thanks, - IL ---- brought to you by your neighborhood Lerxst ----
1	From: guykuo@carson.u.washington.edu (Guy Kuo) Subject: SI Clock Poll - Final Call Summary: Final call for SI clock reports Keywords: SI,acceleration,clock,upgrade Article-I.D.: shelley.1qvfo9INNc3s Organization: University of Washington Lines: 11 NNTP-Posting-Host: carson.u.washington.edu A fair number of brave souls who upgraded their SI clock oscillator have shared their experiences for this poll. Please send a brief message detailing your experiences with the procedure. Top speed attained, CPU rated speed, add on cards and adapters, heat sinks, hour of usage per day, floppy disk functionality with 800 and 1.4 m floppies are especially requested. I will be summarizing in the next two days, so please add to the network knowledge base if you have done the clock upgrade and haven't answered this poll. Thanks.

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 2
documents = newsgroups_data.data[:1000]
df = pd.DataFrame({'text': documents})

documents[0]

stop_words = set(stopwords.words('english'))

print("Sample Stop Words:")
print(list(stop_words)[:5])

def extract_stop_words(text):
    text = text.lower()
    tokens = word_tokenize(text)
    return [word for word in tokens if word in stop_words and word.isalpha()]

stop_words_list = df['text'].apply(extract_stop_words)
stop_words_list
```

```
Sample Stop Words:
['haven', 'yourselves', 'more', 'where', 'should']
```

	text
0	[i, was, if, out, there, me, on, this, i, the,...
1	[a, of, who, their, have, their, for, this, a,...
2	[my, up, the, this, after, as, a, in, i, in, t...
3	[do, you, have, i, to, some, about, this]
4	[from, by, a, my, is, that, the, are, in, the

```
# EXPERIMENT-3B SMuthuraj(241801176)

# Step 3
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)

    tokens = word_tokenize(text)

    tokens = [w for w in tokens if w not in stopwords.words('english')]

    return ' '.join(tokens)

df['clean_text'] = df['text'].apply(preprocess)
df['clean_text']
```

	clean_text
0	wondering anyone could enlighten car saw day d...
1	fair number brave souls upgraded si clock osci...
2	well folks mac plus finally gave ghost weekend...
3	weiteks addressphone number id like get inform...
4	article cowcbnpworldstdcom tombakerworldstdcom...
...	...
995	vt vt compatible terminal external hyess modem...
996	obviously really dont know spoken cf secy stat...
997	noticed exact phenomenon occurs lciii perhaps ...

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 4
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 5
query = "space shuttle launch"
query_clean = preprocess(query)
query_vec = vectorizer.transform([query_clean])
```

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 6
similarities = cosine_similarity(query_vec, tfidf_matrix).flatten()

similarities[:5]
```

```
array([0.        , 0.        , 0.        , 0.        , 0.10156444])
```

```
# EXPERIMENT-3B Muthuraj(241801176)

# Step 7
top_indices = similarities.argsort()[-5:][::-1]

print("\n🔍 Top 5 Relevant Documents for Query:", query)
for i in top_indices:
    print(f"\n📄 Doc {i+1} | Score: {similarities[i]:.4f}\n{df.loc[i, 'text'][:400]}...\n")

    for i in top_indices:
        print(df.loc[i, 'text'][:300])
```

```
    SHUTTLE LAUNCHINGS AND LANDINGS; SCHEDULES AND HOW TO SEE THEM

    Shuttle operations are discussed in the Usenet group sci.space.shuttle,
    and Ken Hollis (gandalf@pr

In fact, you probably want to avoid US Government anything for such a
project.  The pricetag is invariably too high, either in money or in
hassles.

The important thing to realize here is that the big cost of getting to
the Moon is getting into low Earth orbit.  Everything else is practically
down
Archive-name: space/new_probes
Last-modified: $Date: 93/04/01 14:39:17 $

UPCOMING PLANETARY PROBES - MISSIONS AND SCHEDULES

    Information on upcoming or currently active missions not mentioned below
    would be welcome. Sources: NASA fact sheets, Cassini Mission Design
    team, ISAS/NASDA laun
According to a Software engineering professor here, what was actually rated
level five was an ibm unit which produced part of the software for the shuttle,
by not means all of it.
```

```
# EXPERIMENT-3B Muthuraj(241801176)

# POS Tagging
for i in range(3):
    tokens = word_tokenize(df.loc[i, 'clean_text'])
    tagged = nltk.pos_tag(tokens)
    print(f"\n📄 Document {i+1} POS Tags:")
    print(tagged[:10])
```

```
"""
📄 Document 1 POS Tags:
[('wondering', 'VBG'), ('anyone', 'NN'), ('could', 'MD'), ('enlighten', 'VB'), ('car', 'NN'), ('saw', 'JJ'), ('day', 'NN'), ('door', 'VB'), ('sports', 'NNS'), ('car', 'NN')]

📄 Document 2 POS Tags:
[('fair', 'JJ'), ('number', 'NN'), ('brave', 'VBP'), ('souls', 'NNS'), ('upgraded', 'VBD'), ('si', 'JJ'), ('clock', 'NN'), ('oscillator', 'NN'), ('shared', 'VBD'), ('experiences', 'NNS')]

📄 Document 3 POS Tags:
[('well', 'RB'), ('folks', 'NNS'), ('mac', 'VBP'), ('plus', 'CC'), ('finally', 'RB'), ('gave', 'VBD'), ('ghost', 'VBN'), ('weekend', 'NN'), ('starting', 'VBG'), ('life', 'NN')]
```

RESULT:

The information retrieval workflow was executed using the 20 Newsgroups dataset. Text preprocessing and stop-word removal were performed, documents were represented using TF-IDF vectors, and cosine similarity was used to rank documents for the query “space shuttle launch”. The top five relevant documents and sample part-of-speech tags were displayed.